Laboratorio 2

Diego Stefanini — prof. Patrizio Dazzi, Luca Ferrucci, a.a. 2026-27

Indice

1 L'ambiente di lavoro UNIX 3
1.1 Il terminale e il filesystem 3
1.2 Lavorare con i file 4
1.3 Flussi: redirezione e pipe 6
2 Il linguaggio C 8
2.1 Dal sorgente all'eseguibile 8
2.2 Com'è fatto un programma C 10
2.3 Tipi e costanti 12
2.4 Operatori ed espressioni 14
2.5 Il controllo del flusso 16
2.6 Array e tipi custom 19

1 L'ambiente di lavoro UNIX

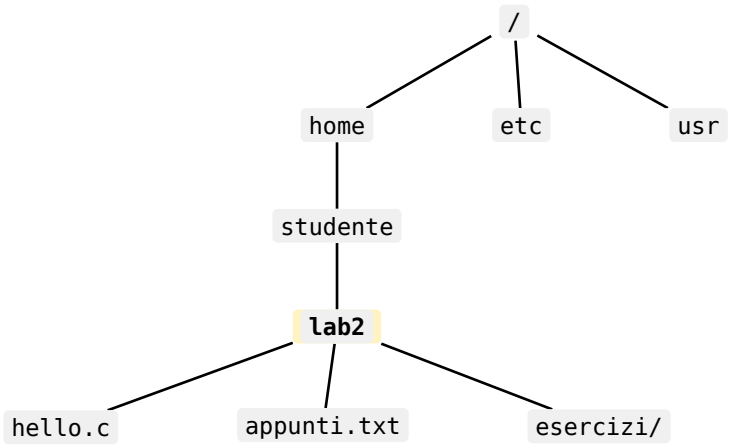
1.1 Il terminale e il filesystem

Si lavora dal **terminale**: una finestra in cui gira la **shell** (bash, zsh), il programma che legge i comandi e li esegue.

```
$ pwd
/home/studente/lab2
$ ls
appunti.txt  esercizi  hello.c
```

Il **prompt** (`$`) dice che la shell aspetta un comando. Non fa parte del comando e non è sempre `$` .

I comandi agiscono sul **filesystem**, un albero di directory e file:



Parte da `/` , la **root**: ogni file o directory si raggiunge con un **percorso**, in cui `/` separa un livello dal successivo.

File e directory sono oggetti diversi, ma nella stessa directory non possono avere lo stesso nome.

In giallo la **directory corrente**.

- `.` = la directory corrente
- `..` = la directory padre

Comando	Descrizione
`pwd`	*print working directory*: stampa in quale cartella sei
`ls`	elenca il contenuto della directory corrente
`ls -l`	elenco in formato lungo (permessi, proprietario, dimensione, data)
`cd`	*change directory*: navigo in una directory
`mkdir`	*make directory*: crea una directory
`echo $?`	restituisce il valore di ritorno dell'ultimo comando eseguito

Per indicare un file nell'albero si usa un **percorso**, assoluto o relativo:

Relativo: parte dalla directory corrente (es. `esercizi/es1.c`).

Assoluto: parte dalla root `/` (es. `/home/studente/lab2`).

```
$ cd esercizi
$ pwd
/home/studente/lab2/esercizi
```

```
$ cd /home/studente
$ pwd
/home/studente
```

```
$ pwd
/home/studente/lab2
$ cd ..          # vado nel padre
$ pwd
/home/studente
```

`~` = la **home** dell'utente corrente. `cd` senza argomenti porta di solito lì.

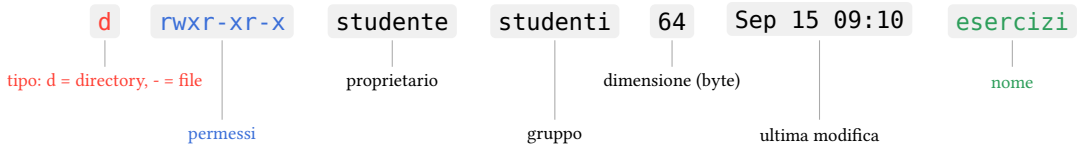
```
$ cd ~
$ pwd
/home/studente
```

Un nome che inizia con `.` è **nascosto**: `ls` non lo mostra, `ls -a` sì.

```
$ ls
appunti.txt  hello.c
$ ls -a
.  ..  .bashrc  appunti.txt  hello.c
```

> «Nascosto» è solo una **convenzione sul nome**, non un tipo speciale di file.

Con `ls -l` si vedono le informazioni su ogni file. Ecco come leggerle:



Ogni comando ha la stessa forma. Per esempio `gcc`, il compilatore C che si usa per tutto il corso:



Le **opzioni** modificano il comportamento del comando. Se non ricordi come si usa: `gcc --help` (sintesi rapida) oppure `man gcc` (manuale UNIX).

Gli argomenti li prepara la shell: **spezza la riga in parole usando gli spazi** e avvia il programma passandogli gli argomenti. Se un nome contiene spazi, le **virgolette** lo tengono insieme come un solo argomento:

```
$ touch "appunti lezione.txt"     # un file, non due
$ cat "appunti lezione.txt"
```

> Vedremo come un programma C legge questi argomenti con `argc` e `argv`, i parametri di `main`.

1.2 Lavorare con i file

Per creare directory e file vuoti si usano `mkdir` e `touch`:

```
$ mkdir esercizi     # argomento = nome
$ touch appunti.txt  # file vuoto
```

`touch` in realtà nasce per **cambiare i timestamp** (gli orari registrati) di un file. Se il file non esiste lo crea: la creazione è una conseguenza utile, non lo scopo.

Oltre al contenuto, per ogni file UNIX tiene tre marcature temporali (**timestamp**). `stat file` le mostra, insieme agli altri metadati.

Metadato	Significato
`atime`	*access time*: ultimo accesso al **contenuto**
`mtime`	*modification time*: ultima modifica del **contenuto**
`ctime`	*change time*: ultima modifica dei **metadati**

`ctime` vuol dire **change** time, **non** «creation time». Alcuni filesystem moderni salvano anche il tempo di creazione (*birth time*), ma non fa parte dei tre timestamp UNIX classici.

A cosa servono: backup incrementali, sincronizzazione, diagnostica, cercare i file modificati di recente, e ricompilare solo ciò che è cambiato. `make` (lo vedremo più avanti) confronta i timestamp per decidere se ricompilare.

Chi aggiorna cosa (✓ = portato all'ora corrente):

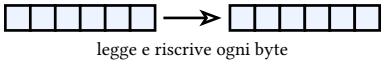
Operazione	`atime`	`mtime`	`ctime`
`touch file`	✓	✓	✓ (cambiare i tempi è già una modifica dei metadati)
`touch -a file`	✓	—	✓
`touch -m file`	—	✓	✓
`cp` : destinazione	✓ solo se creato	✓	✓
`cp -p` : destinazione	copiato dal sorgente	copiato dal sorgente	✓
`cp` : sorgente	✓ può, per la lettura	—	—

Per copiare, spostare e cancellare:

`cp sorgente destinazione` (*copy*): l'originale resta al suo posto.

```
$ cp hello.c copia.c
$ cp /tmp/ciao.txt ~/hello.txt   # copia e rinomina
```

copia — $O(N)$



legge e riscrive ogni byte

sposta (stesso filesystem) — $O(1)$



cambia solo il **percorso** nei metadati

`mv` (*move*): un comando, **due usi**.

```
$ mv copia.c esempio.c      # rinomina
$ mv esempio.c esercizi/    # sposta
```

Operazione	Costo
copia	$O(N)$
sposta, stesso filesystem	$O(1)$
sposta, altro filesystem	$O(N)$: copia + cancella

Comando	Descrizione
`rm`	cancella un file (*remove*)
`rmdir`	cancella una directory **vuota**
`rm -r`	cancella una directory con tutto il contenuto. `-r` = **ricorsivamente**: attraversa tutto ciò che c'è dentro

`rm` **non** sposta nel cestino: da terminale la cancellazione è in genere **definitiva**.

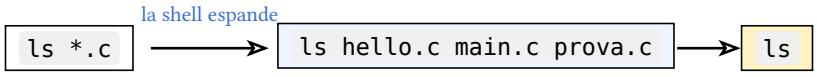
Per guardare dentro un file non serve sempre un editor:

Comando	Descrizione
`cat`	scrive tutto il contenuto del file sul terminale
`less`	per file lunghi: si scorre avanti e indietro e si cerca, senza riversare tutto sul terminale. `q` **per uscire**
`head` / `tail`	prime / ultime 10 righe. `head -n 5 dati.txt` = prime 5

Per lavorare su molti file insieme ci sono le **wildcard**, che espande la shell:

- `*` = **zero o più** caratteri qualsiasi
- `?` = **esattamente un** carattere

```
$ ls *.c
hello.c  main.c  prova.c
$ ls prova?.c
prova1.c  prova2.c  provaA.c
```

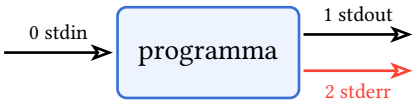


l'espansione avviene **prima** che `ls` parta: `ls` non vede mai l'asterisco

1.3 Flussi: redirezione e pipe

Ogni programma ha tre **flussi standard**:

n.	Flusso	Di solito
0	`stdin` — standard input	dalla tastiera
1	`stdout` — standard output	sul terminale
2	`stderr` — standard error	sul terminale



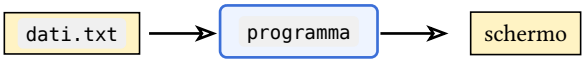
La shell può collegare ogni flusso a qualcosa di diverso dal terminale. Le **redirezioni** lo collegano a un file:



`ls > elenco.txt` — crea o **sovrascrive**



`echo "..." >> log.txt` — **aggiunge** in fondo (*append*)



`programma < dati.txt` — stdin dal file



`gcc errore.c 2> errori.txt` — solo **stderr**

```
$ ls > elenco.txt                # sul terminale non compare niente
$ cat elenco.txt
appunti.txt
elenco.txt
esercizi
hello.c
$ echo "prima riga" > log.txt
$ echo "seconda riga" >> log.txt
$ cat log.txt
prima riga
seconda riga
```

Con `<` il programma continua a leggere da stdin: è **la shell** che ha collegato stdin al file, il programma non se ne accorge.
`2>` redirige il file descriptor 2. Si separa stderr da stdout perché output normale ed errori hanno significati diversi e spesso si trattano in modo diverso.

La **pipe** `|` invece collega lo **stdout del primo** comando allo **stdin del secondo**.



1. la shell espande `*.c`
2. `ls` produce l'elenco
3. la pipe lo passa a `wc`
4. `wc -l` conta le righe

Con un file temporaneo

```
$ ls > tmp.txt
$ wc -l < tmp.txt
```

Con la pipe: niente file in mezzo

```
$ ls | wc -l
$ ls | less      # scorro l'elenco
```

Idea UNIX: piccoli programmi che si combinano per costruire operazioni più complesse.

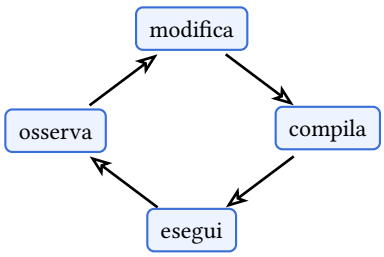
Micro-esercizio. Partendo dalla home: creare `lezione02`, entrarci, creare `a.c`, `b.c`, `note.txt` vuoti, salvare l'elenco dei soli `.c` in `sorgenti.txt`, aggiungerlo a `note.txt`, contare le righe di `note.txt`.

```
$ mkdir lezione02
$ cd lezione02
$ touch a.c b.c note.txt
$ ls *.c > sorgenti.txt
$ cat sorgenti.txt >> note.txt
$ wc -l < note.txt
2
```

2 Il linguaggio C

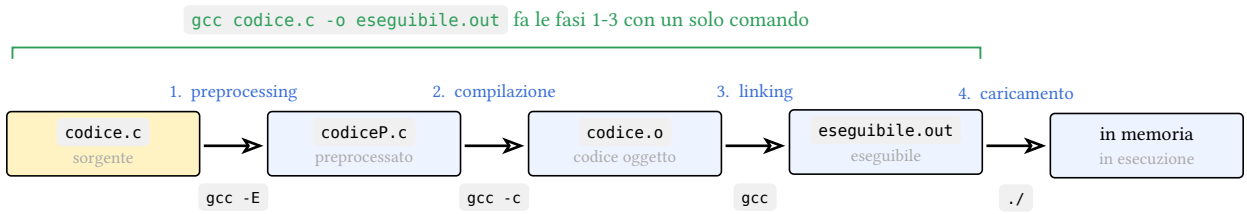
2.1 Dal sorgente all'eseguibile

Scrivere un programma C è un ciclo che si ripete:



> **modifica ≠ compilazione ≠ esecuzione.** Se cambi il sorgente e non ricompili, stai eseguendo il vecchio programma.

Il passo centrale è la compilazione. C è un linguaggio **compilato**: il sorgente `.c` è un semplice file di testo, e per diventare un programma che gira attraversa **quattro fasi**.



1. **Preprocessing**: vengono eseguite le **direttive per il pre-processore**, le righe che iniziano con `#`.
2. **Compilazione**: il sorgente viene tradotto in **codice oggetto** `.o`.
3. **Linking**: i file oggetto vengono collegati in un unico **eseguibile**.
4. **Caricamento**: quando lo lanci, l'eseguibile viene copiato in memoria e parte.

> **Da sapere — codice oggetto e linking** (aggiunto, non spiegato in aula)
> Il codice oggetto è il sorgente già tradotto in linguaggio macchina, ma non ancora eseguibile: gli mancano i pezzi che stanno altrove. Per esempio il codice di `printf` non è nel tuo file, sta nella libreria standard. Il **linker** mette insieme il tuo `.o` e questi pezzi, e ne esce un file che si può eseguire.

La prima fase lavora solo sul testo: le direttive sostituiscono un nome con il contenuto che indica. Due usi:

- `#include <stdio.h>` **include un file**: al suo posto viene incollato il contenuto di una libreria come `stdio.h`, o di un altro file come `my_lib.c`;
- `#define NOME testo` **definisce una macro**: da lì in poi ogni `NOME` viene sostituito con `testo`. Serve per costanti o comandi semplici.

Con `gcc -E` ci si ferma dopo il preprocessing e si vede cosa arriva al compilatore:

```c
#include <stdio.h>
#define CONST_MESSAGE "Hello world\n"

int main(void) {
    printf(CONST_MESSAGE);
    return 0;
}
```

```
$ gcc -E codice.c -o codiceP.c
$ wc -l codiceP.c
845 codiceP.c     # 7 righe diventate 845
$ tail -4 codiceP.c
int main(void) {
    printf("Hello world\n");
    return 0;
}
```

Le righe in più sono il contenuto di `stdio.h`, incollato dall'`include`. In fondo c'è il nostro `main`, con `CONST_MESSAGE` già sostituita dal suo testo.

Di solito le prime tre fasi si fanno con un solo comando, `gcc` (*GNU Compiler Collection*):

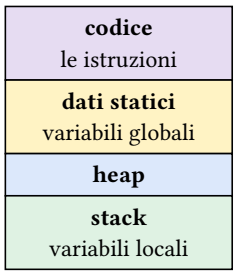
```
$ gcc hello.c              # se va tutto bene non scrive niente
$ ls
a.out  hello.c             # senza -o l'eseguibile si chiama a.out
$ gcc -Wall -Wextra -std=c11 hello.c -o hello   # warning, C11, nome hello
$ ./hello
Hello, world!
$ echo $?
0
```

Pezzo	Significato
`-o hello`	dà il nome all'eseguibile
`-Wall -Wextra`	attiva i warning: **non sono rumore**, spesso segnalano un errore vero
`-pedantic`	segnala tutto ciò che non rispetta alla lettera lo standard del C
`-g`	aggiunge all'eseguibile le informazioni per il *debugger* (il programma che serve a cercare gli errori)
`-std=c11`	standard del C da usare (esistono C89, C95, C99, C11, C17, C23)
`./hello`	«esegui `hello` **che sta nella directory corrente**». La shell cerca i comandi in un elenco di directory, e quella corrente non c'è per forza
`echo $?`	stampa il valore restituito dall'ultimo programma. **0 = terminato correttamente**

Le opzioni complete sono in `man gcc` e su gcc.gnu.org/onlinedocs/gcc. Volendo, le fasi si possono anche separare:

```
$ gcc -E codice.c -o codiceP.c     # 1. solo preprocessing
$ gcc -c codice.c -o codice.o      # 1+2: si ferma al codice oggetto
$ gcc -c codiceP.c -o codice.o     # oppure: compila il file già preprocessato
$ gcc codice.o -o eseguibile.out   # 3. linking
$ ./eseguibile.out                 # 4. caricamento ed esecuzione
Hello world
```

Nella quarta fase l'eseguibile viene caricato in memoria, divisa in quattro zone:



Variabili globali e locali sono spiegate nella prossima sezione.

> **Da sapere — stack e heap** (aggiunto, non spiegato in aula)
> Lo **stack** (pila) cresce e si svuota da solo: quando una funzione parte, le sue variabili locali vengono messe in cima; quando finisce, vengono tolte. L'**heap** è la memoria che il programma chiede esplicitamente mentre gira, e che resta finché non la restituisce.

Se qualcosa va storto, l'errore può arrivare in due momenti diversi:

```c
#include <stdio.h>

int main(void) {
    puts("Hello, world!")   // manca ;
    return 0;
}
```

```
hello.c:4:26: error: expected ';' before
'return'
```

Errore di compilazione	Errore di esecuzione
il programma **non viene costruito**	l'eseguibile esiste ma **si comporta male** o termina in modo inatteso

File, riga, messaggio: usali. Correggi **dall'alto verso il basso**: un solo errore può generare molti messaggi dopo.

Se non funziona, controlla (sono gli errori banali che fanno perdere tempo):
1. sono nella directory giusta? (`pwd`)
2. il sorgente esiste, ed è quello che ho modificato? (`ls`)
3. ho ricompilato? ci sono errori o warning?
4. l'eseguibile è stato creato, e sto eseguendo quello giusto?

2.2 Com'è fatto un programma C

C segue il **paradigma imperativo**: il programma è una sequenza di istruzioni che, una dopo l'altra, cambiano i valori in memoria. Ed è **strutturato a blocchi**: il programma è una collezione di **funzioni**, e l'esecuzione parte sempre dalla funzione `main` . Un file C ha tre parti, in quest'ordine:

```c
#include <stdio.h>   // 1. direttive

int globalA;         // 2. globali

int main(void) {     // 3. funzioni
    puts("Hello!");
    return 0;
}
```

1. **direttive per il pre-processore**: `#include <stdio.h>` rende disponibili le funzioni di input/output (come `puts`);
2. **dichiarazioni globali**: variabili visibili in tutto il programma;
3. **funzioni**: `main` prende niente (`void`) e restituisce un intero (`int`). `return 0` la termina e restituisce 0 al sistema operativo.

Le **graffe** `{ }` delimitano un blocco. Indentazione e nomi sensati non sono obbligatori, ma il programma deve essere leggibile. Per questo servono i **commenti**: testo racchiuso fra `/*` e `*/` (anche su più righe), oppure da `//` a fine riga. Il compilatore li ignora. Documenta **sempre** cosa fa un'istruzione, un blocco o una funzione: serve quando rileggi.

Per stampare ci sono due funzioni, `puts` e `printf` :

Funzione	Cosa fa
`puts("Laboratorio 2");`	stampa una stringa **e va a capo**
`printf("Risultato: %d\n", 42);`	stringa **formattata**: `%d` viene sostituito dal valore. **Non va a capo da sola**: serve `\n`

> `printf` è una funzione a **numero variabile di parametri**: il primo è la stringa di formato, gli altri i valori da inserire al posto dei *placeholder* `%d` , `%s` , …

Mettendo insieme i pezzi:

```c
#include <stdio.h>

int main(void) {
    int x = 21;   // una variabile

    puts("Laboratorio 2");
    puts("Lezione 1");
    printf("2 * %d = %d\n", x, 2 * x);

    return 0;
}
```

```
$ gcc -Wall -Wextra -std=c11
lezione01.c -o lezione01
$ ./lezione01
Laboratorio 2
Lezione 1
2 * 21 = 42
```

I nomi che scegli tu (di variabili, funzioni, tipi) sono **identificatori**: sequenze di lettere, cifre e `_` che iniziano con una lettera o con `_`. Le **parole chiave** del linguaggio (`int`, `return`, `while`, ...) sono riservate e non si possono usare come nomi.

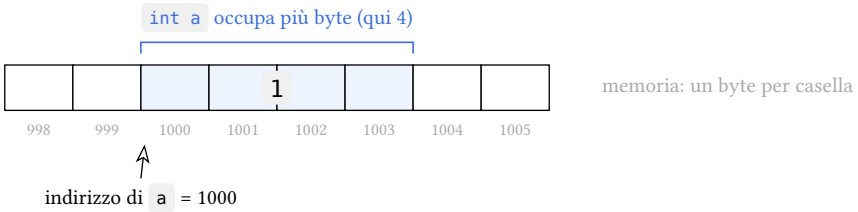
Nome	Valido	Perché
`somma`	sì	lettere
`_tmp`	sì	inizia con `_`
`x1`	sì	la cifra non è all'inizio
`1x`	no	inizia con una cifra
`mia-var`	no	`-` non è ammesso
`int`	no	parola chiave

Nel programma di sopra `x` è una **variabile**: una **locazione di memoria** che contiene un **valore modificabile** di un certo **tipo**. L'insieme dei valori di tutte le variabili in un certo momento è lo **stato del programma**.

```c
int a;        // dichiarazione
int a = 1;    // con valore iniziale
```

La **dichiarazione** dice al compilatore il tipo, e quindi **quanta memoria** riservare.

A ogni variabile è associato un **indirizzo**: un numero intero che indica dove si trova in memoria.



Una variabile però non è utilizzabile ovunque. Lo **scoping** è la regione del programma in cui una variabile è dichiarata, definita e utilizzabile: fuori da lì non è visibile. In C lo scoping è **statico**: viene deciso **a tempo di compilazione**, guardando dove sta la dichiarazione nel testo.

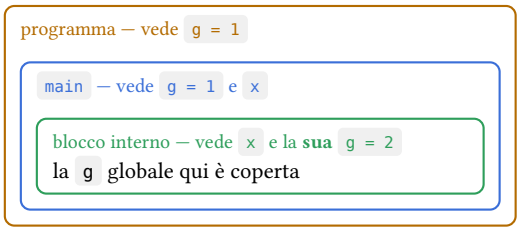
Globale: dichiarata fuori da ogni blocco e funzione. Sta nei **dati statici** ed è visibile nell'intero programma.

Locale: dichiarata dentro un blocco o una funzione. Sta sullo **stack** ed è visibile nel blocco e nei blocchi annidati dentro.

Una variabile locale in un blocco annidato può **coprire** una variabile con lo stesso nome, globale o di un blocco più esterno:

```c
int g = 1;              // globale

int main(void) {
    int x = 5;          // locale a main
    {
        int g = 2;  // copre la g globale
        printf("g = %d, x = %d\n", g, x);
    }
    printf("g = %d\n", g);
    return 0;
}
```




```
$ ./scope
g = 2, x = 5     # dentro il blocco
g = 1            # fuori
```

2.3 Tipi e costanti

C è **strongly typed**: ogni espressione e ogni variabile ha un tipo, noto al compilatore. I tipi di base sono quattro:

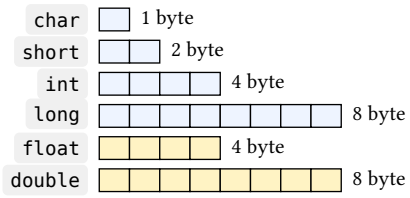
Tipo	Contiene
`char`	un intero grande **un byte**
`int`	numeri interi: `123`, `-45`, `0`; anche costanti carattere come `'a'` o `'1'`
`float`	numeri reali
`double`	numeri reali in **precisione doppia**

Si possono modificare con quattro **qualifier**:

- `short` e `long` qualificano gli interi: `short int a;`. Da soli stanno per `short int` e `long int`. `long` può qualificare anche `double`;
- `signed` e `unsigned` qualificano `char` e `int`: `unsigned` = solo valori ≥ 0.

Quanti byte occupa ogni tipo **dipende dalla macchina**: il C garantisce solo `short` ≤ `int` ≤ `long`. Per saperlo si usa l'operatore `sizeof`, che restituisce la dimensione in byte:

```c
int main(void) {
    int a = sizeof(int);
    printf("%d\n", a);    // stampa 4
    return 0;
}
```



Misurate così su questo computer (Linux a 64 bit):

`sizeof` si applica anche alle **espressioni**, e lì c'è una sorpresa (`%zu` è il placeholder di `printf` per il risultato di `sizeof`):

```c
short int a = 1;
printf("%zu\n", sizeof(a));      // 2
printf("%zu\n", sizeof(a + 1));  // 4
```

Nei calcoli i valori `char` e `short` vengono prima convertiti in `int`. Quindi `a + 1` è un `int`, anche se `a` è `short`: 4 byte, non 2.

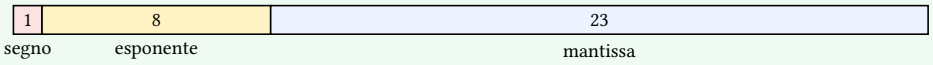
Il `char` è un intero, ma si usa per i **caratteri**. La tabella **ASCII** assegna un carattere a ogni valore che sta in un byte:

'a'	'b'	'*'	'A'	'0'
97	98	42	65	48

Quindi `'a'` e `97` sono lo stesso numero: cambia solo come lo stampi (`%c` o `%d`). Nota `'0'` = 48: il carattere zero non è il numero zero.

> **Da sapere — `'a'` non è un `char`** (aggiunto, non spiegato in aula)
> Una costante carattere come `'a'` in C ha tipo `int`: `sizeof('a')` dà 4, mentre `sizeof(char)` dà 1. Il valore è lo stesso (97), solo che occupa più spazio finché non lo metti in una variabile `char`.

I reali `float` e `double` sono rappresentati in **notazione scientifica**, secondo lo standard **IEEE 754**. Tre componenti: il **bit di segno**, l'**esponente** e la **mantissa**, normalizzata in modo che la parte intera sia 1. Per esempio $1.34423424 \cdot 10^{-5}$ rappresenta 0.0000134423424.

> **Da sapere — quanti bit per ogni parte** (aggiunto, non spiegato in aula)
> In un `float` (32 bit) le tre parti sono grandi così; in un `double` (64 bit) sono 1, 11 e 52 bit.
>
> 
>

Non tutti i numeri sono rappresentabili: i bit sono finiti, quindi molti numeri vengono approssimati al più vicino disponibile. Anche 0.1:

```
$ ./reali                          # printf("%.20f\n", 0.1);
0.10000000000000000555
```

Le **costanti** sono valori non modificabili scritti direttamente nel codice: interi, in virgola mobile, caratteri e stringhe.

Costante	Esempi
intera o in virgola mobile, anche con segno	`1`, `-32`, `3.15`
intera in ottale (inizia con `0`) o esadecimale (inizia con `0x`)	`014`, `0x3A`
carattere, tra apici singoli	`'a'`, `'\n'`
stringa, tra apici doppi	`"ciao!\n"`

Tutte le costanti in virgola mobile sono `double`.

> **Da sapere — ottale ed esadecimale** (aggiunto, non spiegato in aula)
> Sono lo stesso numero scritto in base 8 o in base 16 invece che in base 10. In esadecimale le cifre dopo il 9 sono `A`=10, `B`=11, ..., `F`=15.
> - `014` $= 1 \cdot 8 + 4 = 12$
> - `0x3A` $= 3 \cdot 16 + 10 = 58$
>
> Attenzione: lo `0` iniziale non è decorativo. `010` vale 8, non 10.

Il carattere `\` è un carattere speciale di **escape**: insieme al carattere dopo indica un carattere che non si può scrivere direttamente, perché non stampabile o già usato dalla sintassi.

'\n'	'\t'	'\0'	'\\'

a capo	tabulazione	carattere nullo (codice 0)	la barra `\` stessa

Le costanti stringa sono memorizzate come **array di caratteri**, cioè caratteri uno dopo l'altro in memoria:

> **Da sapere — il terminatore `'\0'`** (aggiunto, non spiegato in aula)
> In fondo a ogni stringa il compilatore aggiunge `'\0'`, che segna dove finisce. Per questo `sizeof("ciao!\n")` vale 7: sei caratteri più il terminatore.

`c`	`i`	`a`	`o`	`!`	`\n`	`\0`

Ogni tipo ha il suo placeholder in `printf`, e **il placeholder deve essere compatibile con il tipo del valore**. Ognuno è fatto così:



`printf("%08.2f", 3.14);` → `00003.14` — forma generale: `%[flags][width][.precision][length]specifier`

Interi		**Altri tipi**	
`%d`, `%i`	int con segno	`%f`	double
`%u`	unsigned int	`%e`, `%E`	notazione scientifica
`%x`, `%X`	esadecimale	`%g`, `%G`	formato compatto
`%o`	ottale	`%c`	carattere
`%ld`	long	`%s`	stringa
`%lld`	long long (ancora più lungo)	`%p`	indirizzo di memoria
`%zu`	`size_t`: il risultato di `sizeof`	`%%`	il carattere %
`%hd`	short	`%Lf`	long double

`printf` scrive su stdout; per **leggere da stdin** c'è `scanf`, sempre nella libreria `stdio`. La stringa di formato è la stessa, ma dopo si passano gli **indirizzi** delle variabili in cui salvare i valori: `&a` è l'indirizzo di `a`.

```c
int a, b;
scanf("%d %d", &a, &b);   // due interi
printf("a = %d\n", a);
```

> I placeholder sono gli stessi di `printf`, **tranne quello per `double`**: in `scanf` è `%lf`.

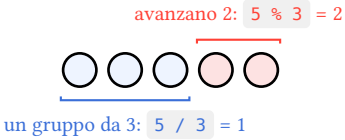
2.4 Operatori ed espressioni

Gli operatori combinano valori e variabili in **espressioni**. Sono di quattro famiglie:

Famiglia	Operatori
aritmetici	`+`, `-`, `*`, `/`, `%`
logici e di confronto	`!`, `&&`, `||` fra valori booleani; `==`, `!=`, `>`, `>=`, `<`, `<=` fra valori numerici
di assegnamento	`=`, `+=`, `-=`, `*=`, `/=`, `%=`
incremento e decremento	`++`, `--`

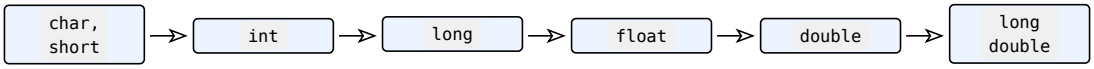
Gli **aritmetici** sono addizione, sottrazione, moltiplicazione, divisione e modulo (il resto della divisione). Il risultato dipende dal tipo degli operandi: tra due interi la divisione è **intera**, la parte decimale si butta.

```
5 / 3        // 1          divisione intera
5.0 / 3.0    // 1.666667   divisione tra reali
5 % 3        // 2          resto
```



Il risultato deve stare nel tipo dell'espressione: se è troppo grande (o troppo piccolo) per i byte a disposizione si ha **overflow** (o **underflow**), e il valore che esce è sbagliato.

Se in un'espressione ci sono tipi diversi, il C li converte da solo (**conversione implicita**): il tipo più «corto» viene trasformato in quello più «lungo», lungo questa scala (è la stessa promozione vista con `sizeof(a + 1)`).



Nell'assegnamento invece **vince il tipo della variabile a sinistra**: se è più corto del valore, si può perdere informazione. Per scegliere noi il tipo c'è la **conversione esplicita**, con l'operatore di **casting**: `(tipo) espressione`. Con `int a = 5;`:

Assegnamento	`b` vale	Perché
`float b = a;`	`5.000000`	l'intero viene promosso a `float`
`float b = a / 2;`	`2.000000`	due interi: divisione **intera**, e solo dopo la conversione
`float b = a / 2.0;`	`2.500000`	`2.0` è reale: `a` viene convertito, divisione tra reali
`float b = (float) a / 2;`	`2.500000`	il casting cambia il tipo di `a` **prima** della divisione

Lo stesso casting serve nei due versi. Per avere il quoziente reale di due interi si converte un operando; per usare `%`, che vuole interi, su un valore letto come `float` lo si converte in `int`:

```
int a, b;
scanf("%d %d", &a, &b);         // 7 2
printf("%d\n", a / b);            // 3
printf("%.2f\n", (float) a / b); // 3.50
```

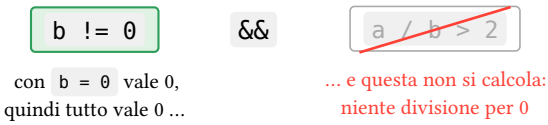
```
float valore;
scanf("%f", &valore);   // 7.0
int n = (int) valore;
if (n % 2 == 0) puts("Pari");
else puts("Dispari");
```

Gli operatori **logici** sono tutti binari tranne `!`. In C **non c'è un tipo booleano di base**: si usano i numeri. **0 è falso, qualsiasi valore diverso da 0 è vero**. Il risultato di un operatore logico o di confronto è sempre **0 se falso, 1 se vero**:

`a`	`b`	`!a`	`a && b`	`a || b`
`0`	`0`	`1`	`0`	`0`

`0`	`-3`	`1`	`0`	`1`
`5`	`0`	`0`	`0`	`1`
`5`	`-3`	`0`	`1`	`1`

`&&` e `||` hanno una **valutazione cortocircuitata**: la valutazione si ferma appena il risultato è noto. Se in `a && b` la `a` vale 0, il risultato è già 0 e `b` non viene nemmeno calcolata; se in `a || b` la `a` è diversa da 0, il risultato è già 1. Così si ottiene un risultato anche quando la parte dopo darebbe errore:



Attenzione a scrivere i confronti come in matematica. `2 < a < 5` **non** controlla che `a` sia tra 2 e 5: equivale a `(2 < a) < 5`, e `2 < a` vale 0 o 1, che è sempre minore di 5. Con `a = 7`:

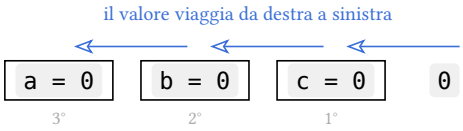
```
a = 7          2 < a < 5
prima 2 < a        1 < 5
risultato      1  (vero!)
```

La forma corretta è `2 < a && a < 5`, che con `a = 7` dà 0. Con `-Wall` il compilatore se ne accorge:

```
warning: comparisons like 'X<=Y<=Z' do not have their
mathematical meaning [-Wparentheses]
```

L'**assegnamento** ha la forma `variabile = parte_destra`. La parte destra è un'espressione, che può essere a sua volta un assegnamento: l'assegnamento stesso restituisce il valore assegnato. Per questo si può scrivere l'**assegnamento multiplo**, che è **associativo da destra**: si esegue partendo dall'ultimo.

```
a = b = c = 0;   // a = (b = (c = 0))
a += 5;          // a = a + 5
a = 3 + 4 * 2;   // 11: prima il calcolo
```



Gli operatori di assegnamento hanno **priorità più bassa** degli altri: nell'ultima riga prima si calcola tutta la parte destra, poi si assegna.

> Attenzione: i tipi devono essere **compatibili**, sia dentro l'espressione sia tra l'espressione e la variabile a cui viene assegnato il valore. Una stringa, per esempio, non diventa da sola un intero: il compilatore dà un warning o un errore.

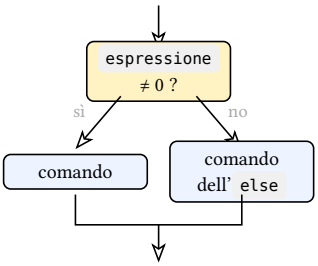
`++` e `--` aumentano o diminuiscono di 1. La posizione conta quando il valore viene usato:

Con `a = 5`	Cosa fa	Risultato
`x = ++a;`	prima incrementa, poi restituisce il **nuovo** valore	`x = 6`, `a = 6`
`x = a++;`	restituisce il valore **attuale**, poi incrementa	`x = 5`, `a = 6`

2.5 Il controllo del flusso

Le istruzioni di un blocco vengono eseguite in ordine. I **comandi** condizionali e i cicli cambiano quest'ordine, e decidono guardando un'espressione: come per gli operatori logici, **diversa da 0 = vera, 0 = falsa**.

```c
if (espressione)
    comando;
else
    comando;
```



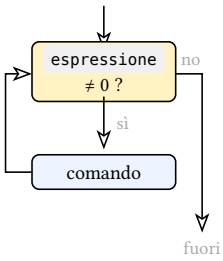
Se l'espressione è diversa da 0 si esegue il primo comando, altrimenti quello dell' `else`. L' `else` si può omettere, e gli `if` si possono annidare uno dentro l'altro.

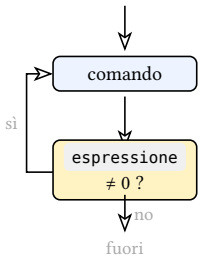
I **cicli** ripetono un blocco di codice finché l'espressione resta diversa da 0; quando diventa 0 si prosegue dopo il ciclo. Il C ne ha tre:

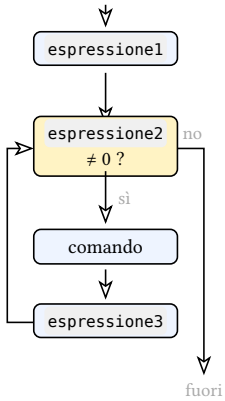
```c
while (espressione)
    comando;
```

```c
do
    comando;
while (espressione);
```

```c
for (espressione1;
     espressione2;
     espressione3)
    comando;
```







Il `while` controlla l'espressione **prima** di ogni giro: se è falsa subito, il corpo non viene mai eseguito. Il `do-while` la controlla **dopo**: il corpo viene eseguito **almeno una volta**. Esempio: stampare le cifre di un numero, dall'ultima alla prima. `n % 10` è l'ultima cifra, `n /= 10` la toglie.

```c
int n = 12345;
while (n > 0) {
    printf("%d\n", n % 10);
    n /= 10;
}
```

`n`	stampa `n % 10`	poi `n` diventa
`12345`	`5`	`1234`
`1234`	`4`	`123`
`123`	`3`	`12`
`12`	`2`	`1`
`1`	`1`	`0`
`0`	`n > 0` falso: fine	

Nel `for`, `espressione1` viene valutata **una sola volta**, prima del primo giro. Poi, finché `espressione2` è vera, si esegue il corpo, e alla fine di ogni giro si valuta `espressione3`. È quindi solo un modo compatto di scrivere un `while`. Ognuna delle tre espressioni si può omettere, come la prima qui a destra:

```c
for (e1; e2; e3)
    comando;
```

```c
e1;
while (e2) {
    comando;
    e3;
}
```

```c
int n = 12345;
for (; n > 0; n /= 10)
    printf("%d\n", n % 10);
```

> Attenzione: un ciclo può **non terminare mai**, se l'espressione non diventa mai 0.

Due istruzioni cambiano il giro dall'interno:

- `break` causa l'**uscita immediata** dal ciclo (o dallo `switch`) che la contiene;
- `continue` **termina il giro corrente** e passa al successivo. Può comparire solo in `for`, `while` e `do-while`.

```c
int a = 0;
while (1) {            // sempre vero
    if (a == 10) break;
    a++;
}                      // qui a vale 10
```

Il ciclo `while (1)` non finirebbe mai: è il `break` che lo chiude.

```c
for (int k = 0; k < 5; k++) {
    if (k == 2) continue;
    printf("%d ", k);
}                      // 0 1 3 4
```

Con `k = 2` il `printf` viene saltato, ma il ciclo va avanti.

Infine lo `switch`, un'istruzione condizionale a **scelta multipla**. L'espressione deve essere di tipo **intero**; ogni `case` ha un'etichetta costante, e le etichette devono essere **uniche**. Il controllo salta all'etichetta uguale al valore dell'espressione, o a `default` se nessuna corrisponde.

```c
switch (espressione) {
    case espr_costante:
        comando;
        break;
    case espr_costante:
        comando;
        break;
    default:
        comando;
}
```

Il `break` qui è essenziale: **senza `break` vengono eseguite anche le istruzioni dei `case` successivi**. Con `c = 2`:

```c
case 1: puts("uno");
case 2: puts("due");   // ← entra qui; niente break: prosegue ↓
case 3: puts("tre"); break;   // → esce
default: puts("altro");
```

Stampa `due` e poi `tre`. Con `-Wextra` gcc avvisa: `this statement may fall through`.

Due esempi che mettono insieme cicli, `if` e `scanf`. A sinistra si legge una sequenza di interi chiusa da 0 (che non ne fa parte) e si contano positivi e negativi: `while`, perché la sequenza può essere vuota. A destra si fa indovinare un numero: `do-while`, perché un tentativo va chiesto **almeno una volta**.

```c
int n, quanti = 0, positivi = 0;
int negativi = 0, somma_positivi = 0;

scanf("%d", &n);
while (n != 0) {
    quanti++;
    if (n > 0) {
        positivi++;
        somma_positivi += n;
    } else {
        negativi++;
    }
    scanf("%d", &n);
}
```

```c
int segreto = 37, tentativo;

do {
    printf("Inserisci un tentativo: ");
    scanf("%d", &tentativo);

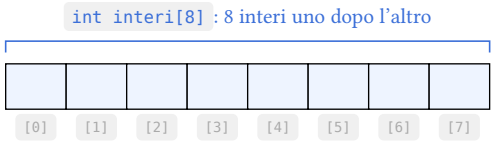
    if (tentativo > segreto)
        printf("Troppo grande\n");
    else if (tentativo < segreto)
        printf("Troppo piccolo\n");
    else
        printf("Indovinato!\n");
} while (tentativo != segreto);
```

2.6 Array e tipi custom

Un **array** è una collezione di valori **tutti dello stesso tipo**. Per definirlo bisogna conoscerne la **dimensione**; alla dichiarazione viene riservato uno spazio **contiguo** in memoria, di solito sullo stack. Si accede a un elemento con l'operatore `[]`: `a[i]`, con `i` che parte da 0.

```c
int interi[8];

#define N 100
double reali[N];   // dimensione con una macro
```



Un array si scorre con un `for` sull'indice. Per esempio il **prodotto scalare** di due vettori, cioè la somma dei prodotti degli elementi con lo stesso indice:

```c
#define N 10
int a[N], b[N], prodotto = 0, i;

for (i = 0; i < N; i++)
    scanf("%d", &a[i]);     // leggo a
for (i = 0; i < N; i++)
    scanf("%d", &b[i]);     // leggo b
for (i = 0; i < N; i++)
    prodotto += a[i] * b[i];
```

`&a[i]` è l'indirizzo della cella `i`: `scanf` scrive lì.

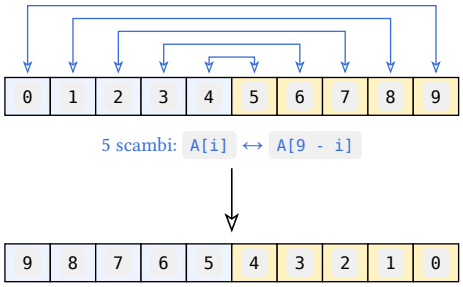
`prodotto` alla fine vale `a[0]*b[0] + … + a[N-1]*b[N-1]`: l'ultimo indice è `N-1`, non `N`.

Secondo esempio: **invertire** un array senza usarne un secondo. Si scambia la prima cella con l'ultima, la seconda con la penultima, e così via **fino a metà**: andando oltre si rimetterebbe tutto com'era.

```c
int i, j, scambio;
int A[10];

for (i = 0; i < 10; i++)
    A[i] = i;

for (i = 0; i < 10 / 2; i++) {
    j = (10 - 1) - i;      // la cella opposta
    scambio = A[i];
    A[i] = A[j];
    A[j] = scambio;
}
```



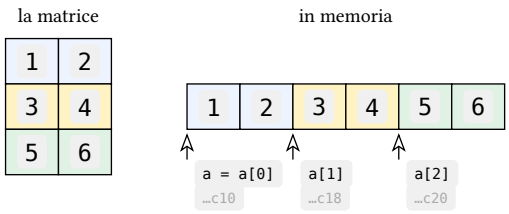
Serve la variabile `scambio` perché `A[i] = A[j]` cancella il vecchio valore di `A[i]`: va messo da parte prima.

Un array può avere più dimensioni. `int a[3][3];` riserva 9 interi contigui, e si usa per rappresentare una **matrice**. In memoria non c'è una griglia: gli array a più dimensioni sono salvati **per righe**, una riga dopo l'altra.

```c
int a[3][2] = {{1,2},{3,4},{5,6}};

printf("%p\n", a);
printf("%p, %p, %p\n", a[0], a[1], a[2]);
```

```
0x7ffd2ba5ac10
0x7ffd2ba5ac10, 0x7ffd2ba5ac18, 0x7ffd2ba5ac20
```



`a`, `a[0]`, `a[1]` e `a[2]` sono gli **indirizzi delle righe**: `a` coincide con `a[0]`, e ogni riga inizia 8 byte dopo la precedente, cioè 2 `int` da 4 byte. Per questo la **seconda dimensione** è importante: per trovare `a[i][j]` bisogna saltare `i` righe intere, e serve sapere quanto è lunga una riga. Le dimensioni possono essere anche più di due: `int a[3][4][2];`.

Una matrice si scorre con due `for` annidati, uno sulle righe e uno sulle colonne. Il prodotto scalare di prima, fra due matrici 10×10:

```c
int a[N][N], b[N][N], prodotto = 0, i, j;
// ... lettura: scanf("%d", &a[i][j]) dentro gli stessi due for
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        prodotto += a[i][j] * b[i][j];
```

Gli array tengono valori dello stesso tipo. Per mettere insieme valori di **tipo diverso** c'è la `struct`. La parola chiave `struct` **definisce un nuovo tipo, non dichiara una variabile**: la variabile si dichiara dopo. I pezzi si chiamano **membri** e si leggono con `variabile.membro`.

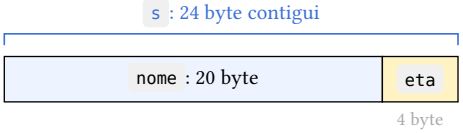
```c
#define LEN 20

struct Studente {     // il nuovo tipo
    char nome[LEN];
    int eta;
};

int main(void) {
    struct Studente s = {"Antonio", 15};

    printf("Nome: %s, Età: %d\n",
           s.nome, s.eta);
    printf("Dimensione: %zu\n",
           sizeof(s));
    return 0;
}
```

```
$ ./main
Nome: Antonio, Età: 15
Dimensione: 24
```



L'ordine dei membri nella definizione è l'ordine in memoria, e lo spazio di una variabile è contiguo. I valori fra graffe vanno **nello stesso ordine dei membri**.

> Con l'ordine sbagliato, `{15, "Antonio"}`, il programma stampa `Nome: T, Età: 0`. Il compilatore avvisa: `incompatible pointer to integer conversion` (con un gcc recente è un errore, e l'eseguibile non viene creato).

> Il compilatore può inserire **padding** fra i campi, cioè byte vuoti, per rispettare vincoli di allineamento: `sizeof` di una `struct` può essere più della somma dei membri.

Il secondo tipo custom è l'`enum`, un tipo **enumerativo**: un elenco di nomi, ognuno dei quali è una **costante intera**. Serve a rendere il codice più leggibile, per esempio nei `case` di uno `switch`. Di default i valori partono da 0 e crescono di 1; si possono cambiare, e da lì in poi si continua a contare.

```c
enum giorni {lunedi, martedi, mercoledi, giovedi, venerdi, sabato, domenica};
```

lunedi	martedi	mercoledi	giovedi	venerdi	sabato	domenica
0	1	2	3	4	5	6

```c
enum giorni {lunedi = 1, martedi, mercoledi, giovedi = 7, venerdi, sabato, domenica};
```

lunedi	martedi	mercoledi	giovedi	venerdi	sabato	domenica
1	2	3	7	8	9	10

Infine `typedef` **dà un nuovo nome a un tipo esistente**: utile per nomi di tipo lunghi, o per averne uno più facile da ricordare. Senza `typedef` le parole `struct` ed `enum` vanno ripetute ogni volta che si dichiara una variabile.

```c
struct Studente { ... };
enum giorni { ... };

struct Studente s;   // senza typedef
enum giorni giorno;
```

```c
typedef struct { ... } Studente;
typedef enum { ... } Giorni;

Studente s;          // con typedef
Giorni giorno;
```

L'esempio che li usa tutti: l'anagrafe di `N` dipendenti. Di ognuno si leggono codice, età, stipendio e ruolo (0 dirigente, 1 quadro, 2 operaio, 3 impiegato); poi si stampano codice, età e ruolo di chi guadagna **meno dello stipendio medio**. I dipendenti stanno in un **array di struct**, il ruolo è un `enum`.

```c
#define N 10

typedef enum {
    dirigente,
    quadro,
    operaio,
    impiegato
} ruolo_t;

typedef struct {
    int id;
    int eta;
    float salario;
    ruolo_t ruolo;
} dipendente_t;
```

```c
dipendente_t dipendenti[N];
int i, ruolo_dip;
float media = 0;

for (i = 0; i < N; i++) {
    scanf("%d", &dipendenti[i].id);
    scanf("%d", &dipendenti[i].eta);
    scanf("%f", &dipendenti[i].salario);
    scanf("%d", &ruolo_dip);
    dipendenti[i].ruolo = ruolo_dip;
    media += dipendenti[i].salario;
}
media /= N;
```

`dipendenti[i].salario` si legge da sinistra: la cella `i` dell'array, e di quella il membro `salario`. La stampa usa uno `switch` sul ruolo, perché l'`enum` è un intero e `printf` stamperebbe solo il numero:

```c
for (i = 0; i < N; i++)
    if (dipendenti[i].salario < media) {
        printf("%d %d ", dipendenti[i].id, dipendenti[i].eta);
        switch (dipendenti[i].ruolo) {
            case dirigente: printf("dirigente\n"); break;
            case quadro:    printf("quadro\n");    break;
            case operaio:   printf("operaio\n");   break;
            default:        printf("impiegato\n");
        }
    }
```

```
3 22 operaio
4 32 impiegato
```